Fruit Into Baskets

1. Problem Title & Link

Title: **LeetCode 904 : Fruit Into Baskets**

Link: https://leetcode.com/problems/fruit-into-baskets/

2. Problem Statement (Clear Explanation)

You are given an integer array fruits where:

fruits[i]

represents the **type of fruit produced by the ith tree**.

You have **two baskets**, and each basket can hold **only one type of fruit**. However, each basket can hold **any quantity** of that type.

You start at any tree and must collect fruit by moving **to the right (forward)** one tree at a time.

Rules:

1. You can only collect fruit from **one continuous segment of trees**.
2. Each basket can store **only one fruit type**.
3. You can carry **at most two different types of fruits**.

Your goal is to determine the **maximum number of fruits you can collect**.

3. Examples (Input → Output)

Example 1

Input

fruits = [1,2,1]

Output

3

Explanation

You can collect all fruits:

[1,2,1]

Two fruit types: 1 and 2.

Example 2

Input

fruits = [0,1,2,2]

Output

3

Explanation

Best segment:

[1,2,2]

Two fruit types.

Example 3

Input

fruits = [1,2,3,2,2]

Output

4

Explanation

Best segment:

[2,3,2,2]

Two fruit types.

Example 4

Input

fruits = [3,3,3,1,2,1,1,2,3,3,4]

Output

5

Explanation

Best segment:

[1,2,1,1,2]

4. Constraints

1 ≤ fruits.length ≤ 100000

0 ≤ fruits[i] < fruits.length

Additional notes:

- Each number represents a **fruit type**
- Only **two distinct fruit types allowed in the window**

5. Core Concept (Pattern / Topic)

Sliding Window

Hash Map / Counting

6. Thought Process (Step-by-Step Explanation)

Step 1 — Understand the objective

We must find the **longest contiguous subarray** containing **at most two distinct numbers**.

Equivalent problem:

Longest subarray with ≤ 2 distinct elements

Step 2 — Brute Force Idea

Check every possible subarray and count fruit types.

This would require:

$O(n^2)$

Too slow.

Step 3 — Sliding Window Idea

Maintain a window with:

at most 2 fruit types

If we exceed two types, move the **left pointer** until the condition becomes valid again.

Step 4 — Data Structure

Use a **hashmap / dictionary** to track the frequency of fruit types inside the window.

7. Visual / Intuition Diagram

Example:

fruits = [1,2,3,2,2]

Window expansion:

[1,2] ✔

[1,2,3] ❌ (3 types)

Move left:

[2,3] ✔

[2,3,2] ✔

[2,3,2,2] ✔

Maximum length:

4

8. Pseudocode (Language Independent)

```
function totalFruit(fruits):

    left = 0
    max_len = 0

    hashmap = {}

    for right in range(len(fruits)):
```

```
    add fruits[right] to hashmap

    while size of hashmap > 2:
        remove fruits[left]
        left += 1

    max_len = max(max_len, right - left + 1)

  return max_len
```

9. Code Implementation

Python

```
def totalFruit(fruits):

  left = 0
  count = {}
  max_len = 0

  for right in range(len(fruits)):

    count[fruits[right]] = count.get(fruits[right], 0) + 1

    while len(count) > 2:

      count[fruits[left]] -= 1

      if count[fruits[left]] == 0:
        del count[fruits[left]]

      left += 1

    max_len = max(max_len, right - left + 1)

  return max_len
```

Java

```
import java.util.*;
```

```
class Solution {

    public int totalFruit(int[] fruits) {

        Map<Integer, Integer> map = new HashMap<>();

        int left = 0;
        int maxLen = 0;

        for(int right = 0; right < fruits.length; right++) {

            map.put(fruits[right], map.getOrDefault(fruits[right], 0) + 1);

            while(map.size() > 2) {

                map.put(fruits[left], map.get(fruits[left]) - 1);

                if(map.get(fruits[left]) == 0)
                    map.remove(fruits[left]);

                left++;
            }

            maxLen = Math.max(maxLen, right - left + 1);
        }

        return maxLen;
    }
}
```

10. Time & Space Complexity

Time Complexity

O(n)

Each element enters and leaves the window once.

Space Complexity

O(1)

At most **3 fruit types stored in map**.

11. Common Mistakes / Edge Cases

Common mistakes:

1. Not shrinking window when **types > 2**
2. Forgetting to **remove fruit type when count becomes 0**
3. Updating max_len incorrectly

Edge cases:

fruits = [1]

fruits = [1,1,1,1]

fruits = [1,2,3,4]

12. Detailed Dry Run

Example:

fruits = [1,2,3,2,2]

Left	Right	Window	Types	Max
0	0	[1]	1	1
0	1	[1,2]	2	2
0	2	[1,2,3]	3	2
1	2	[2,3]	2	2
1	3	[2,3,2]	2	3
1	4	[2,3,2,2]	2	4

Result:

4

13. Common Use Cases (Real-Life / Interview)

Sliding window problems appear in:

- **network traffic analysis**
- **data stream processing**
- **substring problems**
- **real-time monitoring systems**

14. Common Traps (Important!)

1. Using brute force
2. Not shrinking window correctly
3. Forgetting to update counts

15. Builds To (Related LeetCode Problems)

Progression:

LC 3 Longest Substring Without Repeating Characters

LC 904 Fruit Into Baskets

LC 1004 Max Consecutive Ones III

LC 424 Longest Repeating Character Replacement

LC 76 Minimum Window Substring

16. Alternate Approaches + Comparison

Approach	Time	Notes
Sliding Window	O(n)	Optimal
Brute Force	$O(n^2)$	Too slow

17. Why This Solution Works (Short Intuition)

The sliding window ensures we always maintain **at most two fruit types**, expanding when valid and shrinking when invalid.

18. Variations / Follow-Up Questions

Possible interview extensions:

1. Allow **k baskets instead of 2**
2. Return the **actual subarray**
3. Solve with **streaming input**
4. Find **minimum window containing k fruit types**.
5.